United International University

Department of Computer Science and Engineering

Structured Programming Laboratory — Fall 2026

Topic: Basic Introductory Problems in C — Duration: 2.5 hours

Lab Sheet: Basic Introductory Problems

Prepared by: Ashraful Islam Paran, Dept. of CSE, UIU. Follow instructions carefully and complete all assignments within the designated slot.

1. Objectives

By the end of this lab, students will be able to:

- Write, compile, and execute a basic C program.
- Use `printf()` and escape sequences to format output.
- Declare, initialize, and print variables of common C data types.
- Read values from standard input using `scanf()`.
- Distinguish local and global variables and define constants.
- Apply field widths, precision, and scientific notation in formatted output.

2. Introduction to C Programming

C is a procedural, compiled programming language. A basic C program typically includes header files, a `main()` function, statements, and a return value. The `printf()` function displays formatted output, while `scanf()` reads formatted input.

Important notes

- Include `stdio.h` when using `printf()` or `scanf()`.
- Use `\n` for a newline and `\t` for a horizontal tab.
- In `scanf()`, use the address operator `&` for ordinary numeric and character variables.
- Use `%d` for `int`, `%f` for `float` output, and `%lf` to read a `double`.
- C does not have a built-in `boolean` keyword in older standards. Include `stdbool.h` to use `bool`, `true`, and `false`; printing a Boolean as `0` or `1` can be done with `%d`.

3. Common Data Types and Format Specifiers

Type	Typical format specifier	Example
`int`	`%d`	`42`
`float`	`%f`	`3.14f`
`double`	`%f` in `printf`, `%lf` in `scanf`	`3.14159`
`char`	`%c`	`'A'`
`long int`	`%ld`	`100000L`
`long long int`	`%lld`	`100000LL`
`short int`	`%hd`	`12`
`unsigned int`	`%u`	`25U`

The exact ranges of integer types and the precision/range of floating-point types are implementation-dependent. Do not assume that every C compiler supports the same `long double` range.

4. Example 1: Hello World (*)

Task: Write a C program that prints exactly `Hello World`.

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello World\n");
    return 0;
}
```

Output

```
Hello World
```

5. Example 2: Newline and Tab (*)

Task: Write a program that uses newline and tab escape sequences to print the formatted text below.

escape_sequences.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello World.\n");
    printf("\tThis is my first program. C is fun.\n");
    return 0;
}
```

Output

```
Hello World.
    This is my first program. C is fun.
```

6. Example 3: Printing Quotes and a Backslash (*)

Task: Print the following text, including the quotation marks and backslash escape sequence.

quotes_and_backslash.c

```c
#include <stdio.h>

int main(void) {
    printf("The question is - \"How to write a\n\\");
    printf("comment/ in C programming language?\"\n");
    return 0;
}
```

Output

The question is - "How to write a
\comment/ in C programming language?"

7. Example 4: Declaring and Initializing Variables (*)

Task: Declare and initialize an integer, a floating-point number, and a character. Print them using standard field labels. Try at least two sets of values.

variable_types.c

```c
#include <stdio.h>

int main(void) {
    int number = 5;
    float value = 3.141593f;
    char letter = 'a';

    printf("The integer value: %d\n", number);
    printf("The floating point value: %f\n", value);
    printf("The character value: %c\n", letter);
    return 0;
}
```

Output

```
The integer value: 5
The floating point value: 3.141593
The character value: a
```

8. Example 5: Variable Declarations (*)

Task: Write a program that demonstrates: (a) declaring a variable without initializing it, (b) declaring and initializing one variable, (c) initializing multiple variables with different values in one statement, and (d) initializing multiple variables with the same value in one statement. Assign a value to the uninitialized variable before printing it.

declarations.c

```c
#include <stdio.h>

int main(void) {
    int uninitialized;          /* Declared, not initialized */
    int score = 10;             /* Declare and initialize */
    int x = 1, y = 2, z = 3;    /* Different values */
    int a = 7, b = 7, c = 7;    /* Same value */

    uninitialized = 5;          /* Assign before reading */

    printf("uninitialized = %d\n", uninitialized);
    printf("score = %d\n", score);
```

```c
    printf("x = %d, y = %d, z = %d\n", x, y, z);
    printf("a = %d, b = %d, c = %d\n", a, b, c);
    return 0;
}
```

Note: A local variable that is not initialized has an indeterminate value. Assign a value before attempting to read or print it.

9. Example 6: Reading an Integer (*)

Task: Read an integer representing your age from user input and print `My age is:` followed by the value.

read_age.c

```c
#include <stdio.h>

int main(void) {
    int age;

    scanf("%d", &age);
    printf("My age is: %d\n", age);
    return 0;
}
```

Sample input

```
20
```

Sample output

```
My age is: 20
```

10. Example 7: Reading Basic Data Types (*)

Task: Read an integer, a floating-point number, and a character from standard input. Print each value using appropriate format specifiers and labels.

read_basic_types.c

```c
#include <stdio.h>

int main(void) {
    int number;
    float value;
    char letter;

    scanf("%d %f %c", &number, &value, &letter);

    printf("The integer value: %d\n", number);
    printf("The floating point value: %f\n", value);
    printf("The character value: %c\n", letter);
    return 0;
}
```

Note: When reading values on separate lines, whitespace in the format string before `%c` (for example, `"%d %f %c"`) makes `scanf()` skip preceding whitespace.

11. Example 8: Skipping an Input Value (**)

Task: Read three integers but store only the first and third values. Do not assign the middle value to a variable. Print the first and last values.

skip_middle_value.c

```c
#include <stdio.h>

int main(void) {
    int first, last;

    scanf("%d %*d %d", &first, &last);
    printf("First Value = %d, Last Value = %d\n", first, last);
    return 0;
}
```

Note: The assignment-suppression marker `*` in `%*d` reads the middle integer without assigning it to a variable.

12. Example 9: Double and Boolean Values (*)

Task: Declare a `double` and a Boolean value, initialize them, and print the double in scientific notation and the Boolean as `0` or `1`.

double_boolean.c

```c
#include <stdio.h>
#include <stdbool.h>

int main(void) {
    double value = 3.14;
    bool is_valid = true;

    printf("The double value: %e\n", value);
    printf("The boolean value: %d\n", is_valid);
    return 0;
}
```

Output

```
The double value: 3.140000e+00
The boolean value: 1
```

13. Example 10: Signed Data-Type Limits (**)

Task: Print the maximum values of `long int`, `long long int`, `long double`, and `short int`. Use the appropriate standard headers (`limits.h`, `float.h`) and limit macros. Also inspect minimum values where applicable.

integer_ranges.c

```c
#include <stdio.h>
#include <float.h>
#include <limits.h>

int main(void) {
    printf("The long int maximum: %ld\n", LONG_MAX);
    printf("The long long int maximum: %lld\n", LLONG_MAX);
    printf("The short int maximum: %hd\n", SHRT_MAX);
    printf("The long double maximum: %LE\n", LDBL_MAX);
    printf("The long double minimum positive value: %LE\n", LDBL_MIN);
    return 0;
}
```

Note: This version prints limits reported by the compiler and platform rather than hard-coding values. `LDBL_MIN` is the smallest positive normalized value, not the most negative value.

14. Example 11: Unsigned Data-Type Limits (**)

Task: Print the maximum values of `unsigned int`, `unsigned long int`, `unsigned long long int`, and `unsigned short int` using `limits.h`.

unsigned_ranges.c

```c
#include <stdio.h>
#include <limits.h>

int main(void) {
    printf("The unsigned int maximum: %u\n", UINT_MAX);
    printf("The unsigned long int maximum: %lu\n", ULONG_MAX);
    printf("The unsigned long long int maximum: %llu\n", ULLONG_MAX);
    printf("The unsigned short int maximum: %hu\n", USHRT_MAX);
    return 0;
}
```

Note: The standard header `limits.h` provides the maximum values for the corresponding unsigned integer types on the current implementation.

15. Example 12: Defining a Constant with `const` (**)

Task: Define constants for pi and the golden ratio using the `const` keyword. Print pi to two decimal places and the golden ratio to two decimal places.

const_constant.c

```c
#include <stdio.h>

int main(void) {
    const double PI = 3.14;
    const double GOLDEN_RATIO = 1.62;

    printf("The value of pi: %.2f\n", PI);
    printf("The value of golden ratio: %.2f\n", GOLDEN_RATIO);
    return 0;
}
```

16. Example 13: Defining Constants with `#define` (**)

Task: Define `HEIGHT` as 200 and `PI` as 3.14 using preprocessor directives (`#define`). Print both values.

define_constant.c

```c
#include <stdio.h>

#define HEIGHT 200
#define PI 3.14

int main(void) {
    printf("The value of HEIGHT: %d\n", HEIGHT);
    printf("The value of PI: %.2f\n", PI);
    return 0;
}
```

Note: Macro names are replaced by the preprocessor before compilation.

17. Example 14: Global and Local Scope (**)

Task: Declare a global variable and a local variable with the same name but different values. Print (A) the global value before declaring the local variable, (B) the local value, and (C) the global value again. Use a pointer to preserve access to the global variable before it is shadowed.

global_local.c

```c
#include <stdio.h>

int value = 10; /* Global variable */

int main(void) {
    int *global_value = &value; /* Save the global variable's address */

    printf("A. Global: %d\n", value);
    {
        int value = 20; /* Local variable shadows the global */
        printf("B. Local: %d\n", value);
        printf("C. Global: %d\n", *global_value);
    }
    return 0;
}
```

Note: Save the global variable's address in a pointer before declaring the local variable; the pointer can then access the global value.

18. Example 15: Formatted Floating-Point Output (**)

Task: Read a floating-point value and use `printf()` to: (a) right-align it within a minimum field width of 10, (b) print it with a minimum field width of 2, (c) print it rounded to two decimal places, (d) print it rounded to an integer without conversion or type casting, and (e) print it in scientific notation.

formatted_output.c

```c
#include <stdio.h>

int main(void) {
    double value;
    scanf("%lf", &value);

    /* (a) Minimum field width of 10; right-aligned by default */
    printf("(a) Val: %10f\n", value);

    /* (b) Width 2 is a minimum, not a maximum */
    printf("(b) Val:%2f\n", value);

    /* (c) Two digits after the decimal point */
    printf("(c) Val:%.2f\n", value);
```

```c
    /* (d) Print with zero digits after the decimal point */
    printf("(d) Val:%.0f\n", value);

    /* (e) Scientific notation */
    printf("(e) Val: %e\n", value);

    return 0;
}
```

Sample input

```
123.098
```

Sample output

```
(a) Val: 123.098000
(b) Val:123.098000
(c) Val:123.10
(d) Val:123
(e) Val: 1.230980e+02
```

Formatting notes

- `%10f` specifies a minimum field width of 10 characters. If the formatted number is longer, it is not truncated.
- `%2f` does not limit output to two characters; the number is printed in full because the width is only a minimum.
- `%.2f` prints two digits after the decimal point.
- `%.0f` rounds to zero digits after the decimal point using the formatting rules of `printf()`, without an explicit cast.
- `%e` prints scientific notation. Use `%Le` or `%LE` for a `long double`.

Keep Coding!

Strong fundamentals in C are the foundation for learning structured programming and problem solving.